

■ PITCH INSIGHT

Interview Preparation Guide

AI-Powered Cricket Pitch Analyzer

Full-Stack • Deep Learning • Computer Vision • RESTful API

91.6% Accuracy	2,585 Training Images	4 Pitch Classes	6 CV Features	2-4s Process Time
-------------------	--------------------------	--------------------	---------------	----------------------

BACKEND	FRONTEND	ML / AI	DATABASE
FastAPI · Python Uvicorn · Pydantic bcrypt · JWT	React 18 · Vite Axios · Lucide CSS3 Animations	YOLOv8 · ResNet18 OpenCV · PyTorch ONNX Runtime	MongoDB Atlas OpenWeatherMap Razorpay

Prepared on: July 16, 2026 | Covers: Technical, ML/AI, System Design, Behavioral Questions

■ TABLE OF CONTENTS

#	Section	Questions
1	Project Overview & Problem Statement	Questions 1-6
2	System Architecture & Design	Questions 7-12
3	Machine Learning Pipeline	Questions 13-22
4	Computer Vision & Feature Extraction	Questions 23-30
5	Backend — FastAPI & Python	Questions 31-40
6	Frontend — React & Vite	Questions 41-48
7	Database & Authentication	Questions 49-54
8	API Design & Integration	Questions 55-60
9	Deployment & DevOps	Questions 61-65
10	Challenges, Results & Future Work	Questions 66-72
11	Behavioral & HR Questions	Questions 73-80
12	Quick Reference Cheat Sheet	—

■ 1. PROJECT OVERVIEW & PROBLEM STATEMENT

Q1. What is Pitch Insight? Explain it in one sentence.

Pitch Insight is a full-stack AI web application that analyzes cricket pitch images using deep learning (YOLOv8 + ResNet18) and computer vision (OpenCV) to classify pitch conditions and generate real-time match strategies with optional weather integration.

→ Key phrase to remember: 'AI-powered cricket pitch analyzer that classifies conditions and generates match strategies.'

Q2. What problem does Pitch Insight solve?

Before a cricket match, captains and coaches rely on visual inspection and intuition to assess pitch conditions — which is subjective and error-prone.

- Pitch Insight makes pitch analysis objective and data-driven.
- It provides quantifiable metrics: grass %, crack count, moisture score, brightness.
- It integrates live weather to give context-aware match strategies.
- Use case: Pre-match analysis for toss decisions, team selection, batting/bowling order.

Q3. What are the 4 pitch classes your model predicts?

- **Batting Friendly** — Even bounce, minimal seam/spin movement. Great for big scores.
- **Bowling Friendly** — Grass cover aids swing and seam. Bowlers dominate.
- **Spin Friendly** — Cracks and dryness aid spin bowlers. Turn and variable bounce.
- **Seam Friendly** — Hard, dry pitch aids lateral seam movement off the pitch.

→ Memory tip: Batting = balance, Bowling = grass/swing, Spin = cracks/dry, Seam = dry/hard.

Q4. Who are the target users of this application?

- Cricket team coaches & analysts — for pre-match planning.
- IPL/international team management — toss and team selection decisions.
- Cricket broadcast media — real-time pitch analysis for commentary.
- Fantasy cricket platforms — data-driven player selection advice.
- Cricket academies — educational pitch condition understanding.

Q5. What is the overall accuracy of your model, and is it production-ready?

- **Test Accuracy: 91.6% | Validation Accuracy: 91.84%**
- Trained on 2,585 labeled cricket pitch images (70/20/10 split).
- Trained for 30 epochs on Kaggle (GPU T4), took ~20 minutes.
- The gap between validation and test accuracy is minimal (<0.3%), showing good generalization.
- Yes, it is production-ready: ONNX models are deployed for fast cross-platform inference.

Q6. What are the key features of the application?

- AI pitch classification with 91.6% accuracy
- Real-time pitch region detection using YOLOv8
- 6-feature computer vision analysis (grass, cracks, moisture, color, texture, brightness)
- Weather integration via OpenWeatherMap API
- Cricket domain rule engine with 6 expert-defined rules
- Match strategy generator (toss, batting, bowling, team composition)
- User authentication with JWT + bcrypt
- Subscription management with Razorpay payment gateway
- AI cricket chatbot powered by Google Gemini
- Analysis history saved to MongoDB

■ 2. SYSTEM ARCHITECTURE & DESIGN

Q7. Describe the high-level architecture of Pitch Insight.

The application follows a 3-tier client-server architecture:

- **Presentation Layer:** React 18 + Vite frontend (port 3000)
- **Application Layer:** FastAPI + Uvicorn backend (port 8000)
- **Data/AI Layer:** ONNX ML models + MongoDB + External APIs

Communication: REST API with JSON over HTTP/HTTPS.

Frontend sends image as multipart/form-data → Backend processes through ML pipeline → Returns JSON.

Q8. Why did you choose FastAPI over Flask or Django?

- **Async Support:** FastAPI supports async/await natively, crucial for I/O-bound tasks (weather API calls).
- **Automatic Docs:** Swagger UI at /docs and ReDoc at /redoc auto-generated from code.
- **Pydantic Validation:** Strong type checking on request/response models, reduces bugs.
- **Performance:** FastAPI is one of the fastest Python frameworks (comparable to Node.js).
- **Modern Python:** Uses Python 3.10+ features, type hints, dependency injection.

→ *Flask: simpler but no automatic validation. Django: too heavy for API-only use.*

Q9. Why did you use ONNX format for model deployment instead of PyTorch .pth?

- **Speed:** ONNX Runtime is typically 2-3x faster than PyTorch for inference.
- **No framework dependency:** ONNX runs without PyTorch installed, smaller container.
- **Cross-platform:** Same ONNX model runs on CPU, GPU, mobile, edge devices.
- **Memory efficiency:** Lower RAM usage — important for free-tier cloud deployments.
- **Sizes:** PyTorch .pth = 11 MB, ONNX = 44 MB (larger but faster at runtime).

→ *In production, ONNX classifier + ONNX YOLO run on CPU without PyTorch overhead.*

Q10. How does the data flow from user uploading an image to seeing results?

1. User uploads pitch image (JPEG/PNG, max 5MB) via drag-drop or file picker.
2. React creates FormData with image + optional city/lat/long parameters.
3. Axios sends POST /api/analyze (multipart/form-data) to FastAPI backend.
4. FastAPI validates file, saves to temp file, extracts form params.
5. YOLO model detects pitch region → crops to bounding box.
6. OpenCV extracts 6 features from cropped pitch image.
7. ResNet18 classifies cropped image → 4 class probabilities.
8. Rule engine applies 6 cricket domain rules → adjusts probabilities.
9. (Optional) OpenWeatherMap API called if weather enabled.
10. Strategy generator creates toss, batting, bowling recommendations.
11. JSON response sent back → React renders results with animations.

Q11. What design patterns did you use in the backend?

- **Singleton Pattern:** MongoDB connection reused across requests.
- **Factory Pattern:** CompletePitchPipeline creates and manages model instances.
- **Dependency Injection:** FastAPI's Depends() for auth, DB access.
- **Router Pattern:** Modular routers (auth, analysis, chat, subscription, weather).
- **Lifespan Events:** Graceful startup (load models) and shutdown (close DB).
- **Repository Pattern:** database.py abstracts MongoDB operations.

Q12. How does the application handle errors?

- **HTTPException:** FastAPI's built-in exception with status codes (400, 401, 500, 503).
- **Try-Catch blocks:** All ML inference wrapped in try-except.
- **Temp file cleanup:** Finally blocks ensure temp images are deleted even on error.
- **Axios interceptors:** Frontend automatically redirects to login on 401.
- **User-friendly messages:** Generic error messages shown to users, details logged.
- **Common errors:** 400 (not an image), 500 (ML failure), 503 (weather API down).

■ 3. MACHINE LEARNING PIPELINE

Q13. What models does Pitch Insight use and what does each do?

Model 1 — YOLOv8 (pitch_yolov8_best.onnx)

- Task: Object detection — finds the pitch region in a full cricket ground image.
- Output: Bounding box [x1, y1, x2, y2] with confidence score.
- Architecture: Single-stage detector (one forward pass), 640×640 input.

Model 2 — ResNet18 (pitch_classifier.onnx)

- Task: Image classification — classifies the cropped pitch into 4 types.
- Output: Softmax probabilities for 4 classes.
- Architecture: 18-layer residual network, fine-tuned on cricket pitch dataset.
- Input: 224×224 RGB, ImageNet normalized.

Q14. Why did you choose ResNet18 over other architectures?

- **Right size:** ResNet18 has 11M parameters — efficient for this task.
- **Residual connections:** Solve vanishing gradient problem, enable deeper training.
- **Transfer learning:** Pretrained on ImageNet gives excellent visual feature extraction.
- **Proven accuracy:** Achieves 91.6% on our dataset — sufficient for production.
- **Speed:** Inference in ~0.5-0.8s on CPU — acceptable latency.

→ Alternatives considered: ResNet50 (more params, slower), EfficientNet (better but complex), MobileNet (faster but less accurate).

Q15. Explain the ResNet18 training configuration.

- **Dataset:** 2,585 cricket pitch images across 4 classes.
- **Split:** 70% train (1,808), 20% validation (515), 10% test (262).
- **Optimizer:** Adam with lr=0.001.
- **Loss:** CrossEntropyLoss (standard for multi-class classification).
- **Batch size:** 32, **Epochs:** 30.
- **Scheduler:** ReduceLROnPlateau (patience=3, factor=0.5) — reduces LR when validation loss plateaus.
- **Data Augmentation:** Random horizontal flip, rotation $\pm 10^\circ$, color jitter.
- **Platform:** Kaggle GPU T4, training time ~20 minutes.

Q16. What is transfer learning and how did you apply it?

Transfer learning means taking a model pretrained on a large dataset (ImageNet — 1M+ images, 1000 classes) and fine-tuning it on your smaller domain-specific dataset.

How we applied it:

- Loaded ResNet18 pretrained weights from torchvision.
- Replaced the final fully-connected layer: 1000 neurons → 4 neurons (our 4 pitch classes).
- Trained all layers end-to-end (fine-tuning, not feature extraction).
- Benefit: The early layers already detect edges, textures, colors — perfect for pitch features.

→ Without transfer learning, would need ~10x more data to achieve same accuracy.

Q17. What is YOLOv8 and why is it used for pitch detection?

YOLO = 'You Only Look Once' — a real-time single-stage object detector.

- **Single pass:** Unlike two-stage detectors (R-CNN), YOLO processes the whole image once.
- **Speed:** Detects pitch in 0.1–0.2 seconds.
- **Accuracy:** Confidence threshold of 0.5 ensures reliable detections.
- **Custom trained:** Fine-tuned on cricket pitch images with bounding box annotations.

Why needed: A full cricket ground image contains stands, players, grass — we need to isolate just the pitch before classification to avoid noise.

- Output: [x1, y1, x2, y2] bounding box used to crop the pitch region.

Model Performance by Class:

Class	Precision	Recall	F1-Score	Accuracy	Support
Batting Friendly	1.00	0.97	0.98	96.77%	62
Bowling Friendly	0.82	1.00	0.90	100.00%	84
Seam Friendly	0.96	0.93	0.94	92.73%	55
Spin Friendly	0.96	0.74	0.83	73.77%	61
OVERALL	—	—	—	91.60%	262

Q18. Spin-friendly accuracy is only 73.77%. Why? How would you fix it?

Root Cause: Spin-friendly pitches visually resemble seam-friendly pitches (both appear dry/brown). The model confuses them.

Current mitigations:

- Rule engine adds +20% to spin probability when High cracks detected.
- +15% when moisture is Very Dry AND cracks are High.

Future fixes:

- Collect more spin-friendly images (currently 602, lowest along with batting).
- Add crack density as an additional input feature to the classifier.
- Train a binary 'spin vs seam' classifier as a second stage.
- Use class-weighted loss: give spin class higher weight during training.

Q19. Explain softmax and how it's used in your classification.

Softmax converts raw model outputs (logits) into probabilities that sum to 1.

Formula: $\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum(\exp(x_j))}$ for all j

Example output from ResNet18:

- Logits: [1.2, 0.3, 0.8, 3.1]
- After softmax: [batting=5.2%, bowling=8.1%, seam=4.9%, spin=81.8%]
- The model returns spin_friendly with 81.8% confidence.
- Then the rule engine adjusts: if cracks are present → spin += 2% → final 83.7%.

Q20. What data augmentation did you use and why?

- **Random Horizontal Flip:** Pitch looks same from both ends → doubles effective data.
 - **Random Rotation $\pm 10^\circ$:** Camera angle varies in real photos.
 - **Color Jitter** (brightness=0.2, contrast=0.2): Lighting varies by time of day/weather.
 - **Normalization:** ImageNet mean [0.485, 0.456, 0.406] / std [0.229, 0.224, 0.225].
- Augmentation prevents overfitting by making the model learn features, not memorize images.
- We did NOT use vertical flip (pitches have a clear orientation).

Q21. What is the cricket domain rule engine and why is it needed?

Pure ML sometimes misclassifies when lighting or angle is unusual. The rule engine adds cricket expertise as a post-processing correction.

6 Rules (applied after ML inference):

- **Rule 1:** Grass > 60% → Bowling friendly $\pm$ 15% (swing bowling conditions)
- **Rule 2:** High cracks → Spin $\pm$ 20%; Medium cracks → Spin $\pm$ 12%
- **Rule 3:** Very Dry + Grass < 20% → Seam $\pm$ 15%
- **Rule 4:** No/Low cracks + Grass 20-50% → Batting $\pm$ 10%
- **Rule 5:** Very Dry + High cracks → Spin $\pm$ 15% (extreme spin)
- **Rule 6:** Wet/Damp + Grass > 50% → Bowling $\pm$ 12% (humid + grass = swing)

After rules: re-normalize all 4 probabilities to sum to 100%.

Q22. How do you prevent the model from overfitting?

- **Train/Val/Test split:** 70/20/10 — separate test set untouched until final evaluation.
- **Data augmentation:** Random flips, rotations, color jitter.
- **ReduceLROnPlateau:** Reduces learning rate when validation loss stops improving.
- **Early stopping logic:** Model saved at best validation accuracy (not last epoch).
- **Transfer learning:** Pretrained weights provide strong regularization via feature reuse.
- Evidence of no overfitting: Val accuracy (91.84%) $\approx$ Test accuracy (91.60%).

■ 4. COMPUTER VISION & FEATURE EXTRACTION (OpenCV)

Q23. What are the 6 features extracted by OpenCV and what cricket insight does each provide?

1. Grass Coverage (HSV filtering)

- How: Convert BGR→HSV, mask green pixels (H:25-85, S:40-255, V:40-255).
- Output: % of pitch that is green. Levels: High/Medium/Low/Minimal.
- Cricket: More grass = more swing and seam movement for bowlers.

2. Crack Detection (Canny edges)

- How: Grayscale → Gaussian blur → Canny(50,150) → find contours.
- Filter: length >30px, aspect ratio >3:1. Output: crack count, severity.
- Cricket: Cracks = spin turn and variable bounce.

3. Moisture Level (Grayscale intensity)

- How: HSV V-channel mean. Lower brightness = more moisture (0-100 score).
- Levels: Wet/Damp/Normal/Dry/Very Dry.
- Cricket: Wet pitch = slow, low bounce. Dry = turn, crack, bounce.

4. Color Profile (K-means clustering, k=3)

- Types: Brown/Red/Green/Mixed. Indicates pitch preparation type.
- Cricket: Red/Brown = baked clay (India). Green = grass cover (England/NZ).

5. Texture Analysis (Laplacian variance)

- How: Laplacian filter measures edge sharpness = roughness.
- Types: Very Rough/Rough/Moderate/Smooth.
- Cricket: Rough = unpredictable bounce. Smooth = even pace.

6. Brightness (Mean grayscale value)

- 5 levels: Very Dark to Very Bright (0-255 range).
- Cricket: Brightness correlates with pitch hardness and moisture.

Q24. What is HSV color space and why use it over RGB for grass detection?

HSV = Hue, Saturation, Value. RGB = Red, Green, Blue.

- **Hue** encodes the color type (0-180 in OpenCV). Green is H: 25-85.
- **Saturation** encodes color intensity.
- **Value** encodes brightness.

Why HSV over RGB:

- In RGB, green looks different under bright sun vs shade (all 3 channels change).
- In HSV, the Hue stays constant even as brightness changes.
- Simple thresholding: `cv2.inRange(hsv, lower_green, upper_green) → binary mask`.
- Result: Robust grass detection across different lighting conditions.

→ Code: `hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)`

Q25. Explain Canny edge detection and how it finds cracks.

Canny is a multi-step edge detection algorithm:

1. **Grayscale**: Convert color image to single-channel.
2. **Gaussian Blur**: 5×5 kernel — removes noise that would create false edges.
3. **Gradient calculation**: Sobel filters find intensity changes (edges).
4. **Non-maximum suppression**: Thins edges to 1-pixel width.
5. **Double threshold**: Strong edges (>150), weak edges (50-150), no edges (<50).
6. **Edge tracking**: Weak edges kept if connected to strong edges.

Crack filtering: We keep contours where `length>30px` AND `aspect ratio>3:1` (elongated = crack).

→ Short, round blobs are ignored. Long, thin shapes = cracks.

Q26. What is K-means clustering and how is it used for color analysis?

K-means clustering partitions pixels into K groups by minimizing within-cluster variance.

Application in Pitch Insight:

- K=3: Find the 3 dominant colors in the pitch image.
- Each pixel is assigned to nearest color centroid.
- Iteration: centroids updated until convergence.
- Result: Top 3 dominant colors + their percentages.

Then classify: If dominant color is green → 'Green pitch'. If brown/red → 'Brown/Red pitch'.

Cricket use: Green pitch = England-style, Brown = Indian subcontinent, Red = worn clay.

→ `cv2.kmeans(pixels, 3, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)`

Q27. How does the Laplacian variance measure texture/roughness?

The Laplacian is a second-order derivative filter that highlights rapid intensity changes.

- Formula: $L = d^2I/dx^2 + d^2I/dy^2$ (sum of second derivatives).
- High Laplacian variance = lots of edges = rough texture.
- Low Laplacian variance = smooth surface = fewer edges.
- We compute: `var = cv2.Laplacian(gray_image, cv2.CV_64F).var()`

Thresholds used:

- variance > 500 → Very Rough
- variance > 200 → Rough
- variance > 100 → Moderate
- variance ≤ 100 → Smooth

Cricket: Rough pitch = ball grips more, variable bounce (helps spinners and medium-pacers).

Q28. Why do you crop the pitch region before classification?

Without YOLO cropping, the ResNet18 would see the full cricket ground image:

- Includes: spectator stands, outfield grass, advertising boards, umpires.
- These are noise — not relevant to pitch condition.
- The pitch occupies roughly 10-20% of the full image.

Benefits of cropping:

- ResNet18 focuses only on relevant pitch pixels.
- Eliminates false signals from surrounding environment.
- Improves accuracy significantly.
- YOLO confidence threshold of 0.5 ensures only reliable detections are used.

Fallback: If YOLO doesn't detect a pitch (confidence < 0.5), the full image is used.

Q29. How does moisture analysis work technically?

Moisture analysis uses the HSV V-channel (Value = brightness):

- Wet surfaces appear darker (absorb more light).
- Dry surfaces appear brighter (reflect more light).

Steps:

1. Convert image to HSV.
2. Extract V-channel (index 2).
3. Calculate mean brightness: `mean_v = np.mean(v_channel)`.
4. Moisture score = $100 - (\text{mean_v} / 255 * 100)$.
5. Levels: Wet (score>60), Damp (30-60), Normal (20-30), Dry (10-20), Very Dry (<10).

Cricket: Damp pitch = ball moves slower, batsmen comfortable. Very Dry = fast-turning.

Q30. How does weather impact the pitch analysis?

Weather data (from OpenWeatherMap API) adds context to pitch analysis:

- **High Humidity (>70%)**: Ball swings more in air. Impact: Favors bowling.
- **Recent Rainfall**: Wet outfield, slower pitch. Impact: Reduces seam movement.
- **High Temperature (>35°C)**: Pitch dries faster. Impact: More spin later.
- **Strong Wind**: Ball movement in the air. Impact: Helps medium-fast bowlers.

Weather data: temperature, humidity, wind_speed, rainfall, conditions (string).

Severity: low/medium/high based on combination of factors.

→ The strategy generator combines pitch type + weather to give integrated recommendations.

■ 5. BACKEND — FastAPI & PYTHON

Q31. Explain the backend project structure and the role of each file.

- **app.py**: FastAPI initialization, middleware, lifespan events, router registration.
- **config.py**: Environment variables loaded from .env file (API keys, DB URL, secrets).
- **database.py**: MongoDB connection management (singleton pattern).
- **models.py**: Pydantic data models (User, Analysis, Subscription).
- **schemas.py**: API request/response schemas for validation.
- **auth.py**: JWT token creation/verification, bcrypt password hashing.
- **complete_pipeline_onnx.py**: ONNX ML inference pipeline (YOLO + ResNet18).
- **pitch_analyzer.py**: OpenCV feature extraction (PitchAnalyzer class).
- **weather_forecast_analyzer.py**: OpenWeatherMap integration.
- **razorpay_handler.py**: Payment order creation and verification.
- **routes/**: Separate router files — auth, analysis, chat, subscription, weather, health.

Q32. How does JWT authentication work in your application?

JWT = JSON Web Token. Used for stateless authentication.

Flow:

1. User registers → password hashed with bcrypt → stored in MongoDB.
2. User logs in → bcrypt verifies password → JWT token created with user_id + expiry.
3. Token returned to frontend → stored in localStorage.
4. Every subsequent request: Axios adds 'Authorization: Bearer ' header.
5. FastAPI's get_current_user() dependency decodes token → fetches user from DB.
6. If token expired/invalid → 401 Unauthorized returned.

Token: Header.Payload.Signature (base64 encoded, signed with SECRET_KEY).

Expiry: 7 days (configurable).

Q33. What is bcrypt and why is it used for password storage?

bcrypt is a password hashing function specifically designed for security:

- **Slow by design:** Takes 100ms+ to hash — makes brute force attacks impractical.
- **Salt included:** Each hash includes a random salt — no two hashes are same even for same password.
- **Cost factor:** Can increase difficulty as hardware gets faster.
- **One-way:** Cannot reverse the hash to get the original password.

Usage: `pwd_context = CryptContext(schemes=['bcrypt'])`

→ `pwd_context.hash(password)` to hash.

→ `pwd_context.verify(plain_password, hashed)` to verify.

Never store plain text passwords. bcrypt is the industry standard.

Q34. Explain CORS and how it's configured in your application.

CORS = Cross-Origin Resource Sharing. Browser security policy preventing requests between different domains.

Problem: React app (localhost:3000) calls FastAPI (localhost:8000) → different ports = different 'origin'.

Solution: FastAPI CORS middleware tells browser it's OK.

Configuration:

- `allow_origins`: ['http://localhost:3000', 'https://yourdomain.com']
- `allow_credentials`: True (needed for cookies/auth headers)
- `allow_methods`: ['*'] (GET, POST, PUT, DELETE)
- `allow_headers`: ['*'] (Content-Type, Authorization)

In production: Replace '*' with specific domains for security.

Q35. How does file upload work with multipart/form-data?

FastAPI handles file uploads via UploadFile type:

- Frontend creates FormData: `formData.append('image', file)`
- Content-Type: `multipart/form-data` (set automatically by browser)
- Backend: `@app.post('/api/analyze')` with `image: UploadFile = File(...)`

Processing steps:

1. Validate file type: `image.content_type.startswith('image/')`
2. Save to temp file: with `tempfile.NamedTemporaryFile()` as `tmp`: ...
3. Write bytes: `tmp.write(await image.read())`
4. Pass temp path to ML pipeline.
5. Delete temp file in finally block.

→ *python-multipart package required for FastAPI to handle multipart.*

Q36. What is Pydantic and how does it help in FastAPI?

Pydantic is a Python data validation library used heavily by FastAPI.

Uses Python type annotations to validate data automatically.

Benefits:

- **Automatic validation:** Wrong type → 422 Unprocessable Entity response.
- **Serialization:** Python objects → JSON automatically.
- **Documentation:** FastAPI reads Pydantic models to generate Swagger docs.
- **IDE support:** Type hints enable autocomplete.

Example: `class AnalysisResult(BaseModel): prediction: str; confidence: float`

→ *FastAPI uses this for both input (request body) and output (response).*

Q37. How do you manage environment variables and secrets?

- All secrets stored in `.env` file (never committed to Git — listed in `.gitignore`).
- `.env.example` provided with placeholder values for documentation.
- `config.py` reads with `os.getenv()` and `python-dotenv`'s `load_dotenv()`.

Key variables: `MONGODB_URL`, `SECRET_KEY`, `WEATHER_API_KEY`, `GEMINI_API_KEY`,
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`.

In production (Render/Railway): Variables set via platform dashboard, not `.env` file.

→ *Never hardcode API keys in source code. Use environment variables always.*

Q38. Explain the subscription/payment flow with Razorpay.

Razorpay is a payment gateway popular in India.

Flow:

1. User clicks 'Subscribe' → Frontend calls POST /api/subscription/create-order.
2. Backend calls Razorpay API → creates order with amount, currency.
3. Razorpay order_id returned to frontend.
4. Frontend opens Razorpay checkout modal with order_id.
5. User pays → Razorpay calls webhook / returns payment_id to frontend.
6. Frontend sends payment_id + signature to POST /api/subscription/verify-payment.
7. Backend verifies signature using HMAC-SHA256 with Razorpay secret.
8. If valid: Update user's subscription in MongoDB → return success.

Plans: Monthly (■199/30 days), Yearly (■1999/365 days).

Q39. What is the Gemini AI chatbot and how does it work?

The chatbot uses Google Gemini API for conversational AI about cricket.

- User asks questions like: 'What strategy should I use on this pitch?'
- Frontend sends message + current analysis_id to POST /api/chat.
- Backend fetches the analysis result from MongoDB.
- Constructs a context prompt: 'The pitch is spin_friendly with 83% confidence...'
- Sends to Gemini API with cricket-specific system prompt.
- Gemini returns contextual advice.
- Response streamed back to frontend ChatWidget component.

Use case: Natural language Q&A; about the specific pitch that was analyzed.

Q40. How is analysis history stored and retrieved?

Each analysis is saved to MongoDB 'analyses' collection (for logged-in users).

Document structure: {user_id, analysis_id, pitch_type, confidence, features, weather, strategy, timestamp}

- GET /api/analysis/history → returns all analyses for current user.
- GET /api/analysis/{analysis_id} → returns specific analysis.

Frontend HistorySection component displays past analyses in a timeline.

Anonymous users: Analysis not saved (guest mode).

Premium users: Unlimited history. Free users: Last 10 analyses.

■ 6. FRONTEND — REACT & VITE

Q41. Why did you choose React + Vite over other frameworks?

React:

- Component-based architecture: reusable, maintainable UI.
- Virtual DOM: efficient re-rendering only changed components.
- Hooks: useState, useEffect for clean state management without classes.
- Huge ecosystem: Lucide icons, Axios, extensive community support.

Vite:

- Lightning-fast HMR (Hot Module Replacement): instant dev feedback.
- Native ES modules: no bundling during development.
- Faster build than Create React App / Webpack.
- Built-in env variable handling (VITE_API_URL).

Q42. Explain the React component architecture of the application.

App Component (root) manages global state:

- States: currentPage, user, token, theme.
- Renders: Header + current page + Footer.

Components:

- **Header.jsx**: Navigation, auth buttons, theme toggle.
- **UploadSection.jsx**: Drag-drop upload, weather toggle, analyze button.
- **ResultsSection.jsx**: Pitch type card, probabilities chart, features grid, strategy.
- **Auth.jsx**: Login/Signup modal with form validation.
- **ChatWidget.jsx**: Floating chat interface for Gemini AI.
- **PaymentModal.jsx**: Razorpay subscription UI.
- **HistorySection.jsx**: Past analysis timeline.
- **Footer.jsx**: Links to GitHub, API docs.

Q43. How does the image upload and preview work?

- File input (hidden):
 - Drag-drop area: `onClick` triggers `fileInput.click()`.
 - File validation: Checks `file.type.startsWith('image/')` and `file.size < 5MB`.
 - Preview: `URL.createObjectURL(file)` creates a temporary local URL.
 - `img src={preview}` displays the image instantly (no server round-trip).
 - FormData: `formData.append('image', file)` prepares for API upload.
- `URL.createObjectURL()` is revoked after component unmounts to free memory.

Q44. How does Axios work and what are interceptors?

Axios is a promise-based HTTP client for JavaScript.

- Creates API calls: `axios.post('/api/analyze', formData, config)`
- Handles JSON serialization/deserialization automatically.

Interceptors: Middleware that runs before/after every request.

Request Interceptor: Reads JWT token from `localStorage` → adds to Authorization header.

`config.headers.Authorization = `Bearer ${token}``

Response Interceptor: If 401 Unauthorized → clear `localStorage` → redirect to login.

→ Interceptors prevent writing auth header logic in every API call — DRY principle.

Q45. Explain the state management approach in your React app.

Using React's built-in state management (no Redux needed for this scale):

- **useState:** Component-level state (result, loading, error, user, token).
- **useEffect:** Side effects — load token from `localStorage` on mount.
- **Props drilling:** User and token passed from App → child components.
- **localStorage:** Persist auth state across page refreshes.

State flow: User logs in → `setUser(data)` + `setToken(token)` → `localStorage.setItem()`

On refresh: `useEffect` reads `localStorage` → restores auth state.

When to use Redux: Only needed when 5+ components share the same state — not needed here.

Q46. How did you implement the dark theme and animations?

Dark Theme:

- CSS variables (custom properties) in index.css:

--primary: #10b981, --background: #0f172a, --surface: #1e293b

- Glassmorphism: backdrop-filter: blur(10px) on card components.

Animations:

- fadeIn: opacity 0→1 + translateY(20px→0)

- slideInLeft/Right: translateX(-50px→0) / (50px→0)

- pulse: scale(1)→scale(1.05)→scale(1) (heartbeat on loading icons)

- spin: 360° rotation for loading spinners

- Staggered delays: animation-delay: 0.1s, 0.2s, 0.3s for sequential appearance.

→ All animations use CSS @keyframes, no JavaScript animation libraries.

Q47. How is the application made responsive for mobile?

Mobile-first CSS approach with media query breakpoints:

- Desktop: 1024px+ — multi-column grid layout.

- Tablet: 768px-1023px — 2-column grid.

- Mobile: <768px — single column stack.

Techniques:

- CSS Grid with grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))

- Flexbox for row/column switching at breakpoints.

- Reduced padding/font sizes on mobile.

- Touch-friendly buttons (min-height: 44px for tap targets).

- Drag-drop area works with touch events on mobile.

Q48. How is the environment/API URL configured for different environments?

Vite uses environment variable files:

- Development: VITE_API_URL=http://localhost:8000 (in .env.local)
- Production: VITE_API_URL=https://pitch-insight-api.onrender.com

Access in code: `import.meta.env.VITE_API_URL`

Usage in api.js:

```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000'
```

On Vercel: Add VITE_API_URL as an environment variable in dashboard.

→ *This allows same codebase to work in development and production.*

■ 7. DATABASE & DATA MANAGEMENT

Q49. Why MongoDB over a relational database like PostgreSQL?

- **Schema flexibility:** Analysis results have varying fields (some have weather, some don't).
- **JSON-native:** Analysis results are already JSON — no ORM mapping needed.
- **Easy scaling:** MongoDB Atlas auto-scales with usage.
- **No migrations:** Add new fields without altering schema.
- **Good for document storage:** Each analysis is a self-contained document.

When PostgreSQL would be better: Complex relationships, transactions, strict schema needed.

MongoDB Atlas: Managed cloud MongoDB — free tier available, automatic backups.

Q50. What are the MongoDB collections and their structure?

users collection:

```
{ _id, email, hashed_password, name, created_at, subscription_status, subscription_expiry }
```

analyses collection:

```
{ _id, user_id, analysis_id (string), pitch_type, confidence, features{}, weather{}, strategy{}, timestamp }
```

Indexing:

- users: email (unique index) for fast lookup during login.
- analyses: user_id (index) for fast history retrieval.
- analyses: analysis_id (unique index) for lookup by ID.

Q51. Is there a database for the ML models? How are they stored?

ML models are stored as files on the local filesystem / server:

- pitch_yolov8_best.onnx (12 MB) — YOLO detector
- pitch_classifier.onnx (44 MB) — ResNet18 classifier

Models are loaded into memory once at application startup (lifespan event).

No database needed for models — they are static files that don't change at runtime.

In production (Render): Models are part of the Docker image / repository.

→ Alternative: Store on S3/GCS and download on startup — better for large models.

Q52. How are temporary image files managed to avoid disk space issues?

- Python's `tempfile.NamedTemporaryFile()` creates a temp file with auto-cleanup.
- Files saved in `/tmp` directory (Linux/Mac) or OS temp folder (Windows).
- try-finally pattern ensures deletion even if ML pipeline throws an exception:

```
try: result = pipeline.analyze(tmp.name)
```

```
finally: os.unlink(tmp.name) # Always runs
```

- The stateless design means no permanent image storage — only metadata in MongoDB.

→ For a production system, could add S3 storage if users want to re-analyze images.

Q53. Explain the singleton database connection pattern.

A singleton ensures only one MongoDB connection is created and reused:

```
mongodb_client: Optional[MongoClient] = None
```

```
def get_database():
```

```
    global mongodb_client
```

```
    if mongodb_client is None:
```

```
        mongodb_client = MongoClient(MONGODB_URL)
```

```
    return mongodb_client[DATABASE_NAME]
```

Benefits:

- Avoids creating new connections for every request (expensive operation).
- Connection pooling: MongoClient maintains a pool of reusable connections.
- Graceful shutdown: `close_database_connection()` called in lifespan shutdown.

Q54. How does subscription status affect the user experience?

Free users:

- Can analyze 3 pitches per day.
- Last 10 analyses in history.
- Basic strategy recommendations.

Premium users (after Razorpay payment):

- Unlimited analyses.
- Full analysis history.
- Advanced strategy with detailed weather impact.
- AI chatbot access.

Implementation:

- subscription_status field in users collection: 'free' / 'premium'.
- subscription_expiry: datetime — backend checks if subscription is still active.
- Frontend shows UpgradePrompt component when free limit reached.

■ 8. API DESIGN & INTEGRATION

Key API Endpoints:

Method	Endpoint	Auth	Description
GET	/api/health	None	Health check — is server running?
GET	/api/classes	None	List pitch classes with descriptions
POST	/api/quick-analyze	Optional	Fast classification (0.8-1.5s)
POST	/api/analyze	Optional	Full analysis with features + weather (2-4s)
GET	/api/weather	None	Fetch weather by lat/lon or city
POST	/api/auth/signup	None	Register new user
POST	/api/auth/login	None	Login → returns JWT token
GET	/api/auth/me	JWT	Get current user profile
GET	/api/analysis/history	JWT	User's past analyses
POST	/api/chat	JWT	Send message to Gemini chatbot
POST	/api/subscription/create-order	JWT	Create Razorpay order
POST	/api/subscription/verify-payment	JWT	Verify and activate subscription

Q55. What is the difference between quick-analyze and complete analyze?

POST /api/quick-analyze:

- Only YOLO detection + ResNet18 classification.
- No OpenCV features, no weather, no strategy.
- Response time: 0.8-1.5 seconds.
- Use case: When you just need the pitch type fast.

POST /api/analyze:

- Full pipeline: YOLO + OpenCV features + ResNet18 + rules + weather + strategy.
- Response time: 2-4 seconds (4+ with weather API).
- Returns comprehensive JSON with all features and match strategy.
- Use case: Pre-match full analysis.

Q56. How is the API documented? How can someone test it?

FastAPI auto-generates interactive API documentation:

- **Swagger UI:** <http://localhost:8000/docs> — interactive, try endpoints directly.
- **ReDoc:** <http://localhost:8000/redoc> — clean, readable documentation.

Both are generated from Pydantic models and function type annotations.

Testing methods:

- Swagger UI: Upload image file, test all endpoints in browser.
- curl: `curl -X POST http://localhost:8000/api/quick-analyze -F 'image=@pitch.jpg'`
- Python requests library: programmatic testing.
- `test_api.py`: Our own test script for API validation.

Q57. What HTTP status codes does your API use?

- **200 OK:** Successful request.
- **400 Bad Request:** Invalid input — file is not an image, missing required field.
- **401 Unauthorized:** Missing or invalid JWT token.
- **403 Forbidden:** Valid token but no permission (e.g., free user accessing premium).
- **404 Not Found:** Analysis ID not found in database.
- **422 Unprocessable Entity:** Pydantic validation failed (wrong data type).
- **500 Internal Server Error:** ML pipeline failure, unexpected exception.
- **503 Service Unavailable:** Weather API is down, cannot fetch weather data.

Q58. How does the OpenWeatherMap API integration work?

OpenWeatherMap provides free weather data via REST API.

Function: `get_weather_data(latitude, longitude, city)`

API call: `GET https://api.openweathermap.org/data/2.5/weather`

`?lat={lat}&lon={lon}&appid={WEATHER_API_KEY}&units=metric`

Response parsed into WeatherData model:

`{ temperature, humidity, wind_speed, rainfall, conditions, location }`

Fallback: If lat/lon not provided → use city name parameter instead.

Error handling: If API fails → return None, analysis continues without weather.

→ *Weather is always optional — the pitch analysis works without it.*

Q59. What is the analysis_id format and how is it generated?

analysis_id is a unique string identifier for each analysis.

Format: 'PITCH_YYYYMMDD_HHMMSS'

Example: 'PITCH_20251217_143052'

Generated with: `datetime.now().strftime('PITCH_%Y%m%d_%H%M%S')`

Use cases:

- Retrieve specific analysis later.
- Reference in chatbot context.
- Share analysis link with team.

In MongoDB: Stored as string, indexed for fast lookup.

Q60. How is processing time measured and why is it important?

Processing time is measured per request:

```
start_time = time.time()
```

```
# ... all ML processing ...
```

```
processing_time = time.time() - start_time
```

Typical breakdown:

- YOLO detection: 0.1-0.2s
- OpenCV features: 0.3-0.5s
- ResNet18 classification: 0.5-0.8s
- Rule engine: <0.1s
- Weather API: 0.5-1.0s (if enabled)
- Total: 0.8-1.5s (quick) or 2-4s (full with weather)

Why it matters: User experience — >4s feels slow. Monitoring API performance.

■ 9. DEPLOYMENT & DEVOPS

Q61. How would you deploy this application to production?

Backend → Render (or Railway):

- render.yaml defines service type, build command, start command.
- Build: `pip install -r requirements.txt`
- Start: `uvicorn app:app --host 0.0.0.0 --port $PORT`
- Environment vars set in Render dashboard.
- URL: `https://pitch-insight-api.onrender.com`

Frontend → Vercel (or Netlify):

- Build command: `npm run build` → output: `dist/` folder.
- Vercel auto-detects Vite projects.
- Set `VITE_API_URL` to backend URL.
- URL: `https://pitch-insight.vercel.app`

Database → MongoDB Atlas:

- Cloud-managed MongoDB. Free tier: 512MB.
- Connection string in `MONGODB_URL` env var.

Q62. How would you containerize this application with Docker?

Backend Dockerfile:

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ['uvicorn', 'app:app', '--host', '0.0.0.0', '--port', '8000']
```

Frontend Dockerfile (multi-stage):

Stage 1 (build): `node:18-alpine` → `npm run build` → `dist/`

Stage 2 (serve): `nginx:alpine` → serve `dist/` on port 80.

Benefits: Consistent environment, easy scaling, works anywhere Docker runs.

Q63. What are the render.yaml and Procfile files for?

render.yaml: Infrastructure-as-code for Render platform.

Defines: service name, runtime (python), build command, start command, env vars.

Render reads this file to auto-configure the deployment.

Procfile: Used by Heroku and similar platforms.

Content: web: uvicorn app:app --host 0.0.0.0 --port \$PORT

Defines: process type (web) and the command to start it.

\$PORT is set by the platform automatically.

Both files enable one-click deployment on respective platforms.

Q64. What are the production considerations for a real deployment?

Security:

- HTTPS only (SSL/TLS via platform or nginx).
- Rate limiting (e.g., 10 requests/minute per IP).
- Input validation + file size limits (5MB).
- SECRET_KEY must be a strong random string.

Performance:

- Multiple Uvicorn workers: `gunicorn -w 4 -k uvicorn.workers.UvicornWorker app:app`
- CDN for frontend static files.
- Model caching: load once at startup, reuse.

Monitoring:

- Sentry for error tracking.
- Health check endpoint (`/api/health`) for uptime monitoring.
- Render/Railway provide basic metrics dashboard.

Q65. How would you scale this application if traffic increases 10x?

Horizontal scaling:

- Multiple API server instances behind a load balancer.
- Stateless design (no server-side sessions) → easy to scale horizontally.
- MongoDB Atlas scales automatically with usage.

Vertical scaling:

- GPU inference: Replace CPUExecutionProvider with CUDAExecutionProvider.
- GPU reduces ResNet18 inference from 0.5s to ~0.05s.

Caching:

- Redis cache for identical image analysis (hash image → cache result).
- CDN for frontend assets.

Async processing:

- For very high load: queue analysis jobs in Celery + Redis.
- Return job_id immediately, poll for results.

■ 10. CHALLENGES, RESULTS & FUTURE WORK

Q66. What was the biggest technical challenge you faced?

Challenge 1: Memory optimization for ONNX models on free-tier servers

- Problem: Render free tier has 512MB RAM. YOLOv8 + ResNet18 together used ~800MB.
- Solution: ONNX models use less RAM than PyTorch. Lazy loading. CPU-only inference.
- Created MEMORY_OPTIMIZATION.md with detailed memory profiling.

Challenge 2: Spin vs Seam misclassification

- Problem: Both look similar (dry, brown). Model confused them at 73% accuracy for spin.
- Solution: Added rule-based adjustments using crack detection as a differentiator.

Challenge 3: CORS in development

- Problem: React (3000) calling FastAPI (8000) → CORS blocked.
- Solution: Configured CORSMiddleware with correct origins.

Q67. What would you do differently if you were to rebuild this project?

- Use GraphQL instead of REST for flexible data fetching.
- Use React Query or SWR for server state management and caching.
- Implement proper CI/CD pipeline from day one (GitHub Actions).
- Use Docker Compose for local development (no manual setup).
- Add comprehensive unit tests (pytest for backend, Jest for frontend).
- Use a proper logging library (structlog or loguru) instead of print().
- Train with more data — 2,585 is relatively small for computer vision.
- Use a more modern model: EfficientNetV2 or Vision Transformer (ViT).

Q68. What are the planned future enhancements?

ML Improvements:

- Multi-model ensemble for higher accuracy.
- Time-series model for pitch deterioration prediction (Day 1 vs Day 5).
- Venue-specific models (different pitches behave differently at each ground).

Features:

- Video analysis: Analyze pitch condition from match footage.
- Player performance predictions based on pitch type.
- Historical data integration (ESPNcrinfo API).
- Mobile app (React Native).

Technical:

- WebSocket for real-time updates.
- Kubernetes for auto-scaling.
- GraphQL API.
- Automated testing with pytest + GitHub Actions CI/CD.

Q69. What are the limitations of your current approach?

- **Image quality dependent:** Poor lighting, blurry images reduce accuracy.
- **Static analysis:** Can't track pitch deterioration over match overs.
- **Training data bias:** Most data from professional matches — amateur pitches may differ.
- **Weather is supplementary:** Weather doesn't directly change ML prediction, only strategy.
- **No ground truth validation:** We don't know actual match outcomes to validate strategy.
- **Single image:** Taking one photo may not capture the whole pitch condition.
- **Spin/Seam confusion:** 73% accuracy for spin class needs improvement.

Q70. How would you evaluate if the system is actually useful in practice?

- **Collect feedback:** Ask coaches/analysts if recommendations matched actual pitch behavior.
- **Compare to expert analysis:** Compare our predictions to official pitch curators' reports.
- **Match outcome correlation:** Did teams that followed our toss advice win more?
- **A/B testing:** Compare strategy outcomes for teams using vs not using the tool.
- **Track accuracy over time:** Log predictions vs actual pitch behavior after each match.

→ Currently, the 91.6% accuracy is on a labeled test set, not real-world outcome validation.

Q71. Can your model handle night match conditions or different pitch types globally?

Night matches (floodlights):

- Floodlights change color temperature → may affect HSV grass detection.
- The ML model may not handle artificial lighting well (not in training data).
- Mitigation: Add artificial lighting images to training set.

Global pitch types:

- England (green, grassy) vs India (dry, spin) vs Australia (hard, bouncy).
- Our model is trained on diverse images but may favor more common conditions.
- Venue-specific models would handle this better.
- Current approach: rule engine adjustments help somewhat.

→ *This is a valid limitation to acknowledge in interviews.*

Q72. How do you measure the model's confidence and when is it reliable?

- Confidence = the softmax probability of the predicted class.
- High confidence (>80%): Model is very sure. E.g., spin_friendly at 99.93%.
- Medium confidence (60-80%): Reasonable prediction, check features for context.
- Low confidence (<60%): Model is uncertain — features and rule engine are more important.

Best practices:

- Set a confidence threshold of 80% for 'high confidence' label.
- If confidence is low, show all 4 probabilities prominently.
- After rule adjustment, confidence may increase or decrease.
- Perfect pitch images (clear, well-lit, close-up) → higher confidence.

■ 11. BEHAVIORAL & HR QUESTIONS

Q73. Why did you build this project?

- Cricket is deeply personal — I grew up watching matches and understanding pitch conditions.
- I wanted to apply AI/ML to a real-world domain problem I genuinely care about.
- The project challenged me to integrate multiple technologies: deep learning, computer vision, REST APIs, React, and payment gateways.
- It's a complete full-stack product, not just a model — it demonstrates end-to-end software development.
- I also wanted to solve the subjectivity problem in pitch analysis — make it objective and data-driven.

Q74. What did you learn from building this project?

Technical learnings:

- ONNX model deployment: How to convert PyTorch → ONNX and optimize for production.
- FastAPI's dependency injection and lifespan management.
- OpenCV image processing techniques (HSV, Canny, Laplacian, K-means).
- JWT authentication end-to-end implementation.
- Payment gateway integration (Razorpay webhook verification).

Soft learnings:

- Breaking complex systems into manageable modules.
- Importance of documentation — TECHNICAL_DOCUMENTATION.md helped me debug myself.
- How small model improvements (rule engine) can significantly boost user trust.

Q75. How did you handle a situation where something didn't work as expected?

The ONNX conversion issue:

- Original plan: Use PyTorch .pth model directly.
- Problem: PyTorch + CUDA = 2GB+ dependencies, too heavy for Render free tier.
- Action: Researched ONNX Runtime as a lighter alternative.
- Implementation: Converted models, rewrote inference code with onnxruntime.InferenceSession.
- Result: Memory reduced by ~60%, deployment succeeded on free tier.

Lesson: Always profile memory/performance before choosing a technology for production.

Q76. How do you stay updated with the latest in ML/AI?

- Follow Hugging Face, Papers With Code for latest research.
- Practice on Kaggle competitions.
- YouTube: Andrej Karpathy, Sentdex, Two Minute Papers.
- GitHub: Follow repositories of PyTorch, Ultralytics (YOLO), FastAPI.
- Build projects: Best way to learn is to implement new concepts.

→ For this project: Learned YOLOv8 from Ultralytics docs, ResNet from PyTorch tutorials.

Q77. How would you explain this project to a non-technical person?

'Imagine you're a cricket captain and you need to decide whether to bat or bowl after winning the toss. Traditionally, you'd look at the pitch and use experience to guess its nature.

Pitch Insight is like a smart assistant — you take a photo of the pitch and it analyzes it using AI. It tells you: this pitch will spin a lot, so bat first and score big before the spinners dominate. It also checks the weather and says: humidity is high, which helps swing bowlers.

Think of it as having a cricket expert in your pocket — one that uses data and AI instead of gut feeling.'

Q78. What would you prioritize if you had 2 more weeks to work on this?

Priority 1: Improve spin-friendly accuracy (currently 73.77%)

- Collect 500+ more spin-friendly training images.
- Re-train with class-weighted loss.

Priority 2: Add comprehensive unit tests

- pytest for backend API endpoints and ML pipeline.
- Jest + React Testing Library for React components.

Priority 3: CI/CD pipeline

- GitHub Actions: auto-test on PR, auto-deploy on merge to main.

Priority 4: Video analysis

- Analyze pitch from 5-second video instead of single photo.
- Average predictions across frames for more robust classification.

Q79. What is your biggest takeaway from this project for your career?

- Full-stack AI products require skills beyond just training models.
- Production ML is very different from notebook ML.
- System design matters: a well-designed architecture saved hours of debugging.
- Domain knowledge amplifies ML — the rule engine doubled user trust.
- Documentation is code: good docs made the project maintainable.
- Solved a real problem I care about — passion drives quality.

Q80. Do you have any questions for us? (What to ask the interviewer)

- 'What ML/AI tech stack does your team primarily use?'
- 'How does your team handle model versioning and deployment?'
- 'What's the ratio of research/experimentation vs production work?'
- 'How does the team stay updated with fast-moving ML developments?'
- 'What would a typical project look like for someone in this role?'

→ Always ask a question — it shows curiosity and preparation.

■ 12. QUICK REFERENCE CHEAT SHEET

■ Key Numbers to Remember:

Metric	Value
Test Accuracy	91.60%
Validation Accuracy	91.84%
Training Images	2,585 images
Pitch Classes	4 (batting/bowling/seam/spin)
Training Epochs	30
Batch Size	32
Learning Rate	0.001 (Adam)
ResNet18 Parameters	11,178,564
Model Size (.pth)	11 MB
ONNX Model Size	44 MB
Processing Time (quick)	0.8-1.5 seconds
Processing Time (full)	2-4 seconds
YOLO Confidence Threshold	0.5
OpenCV Features Count	6
Domain Rules Count	6
ResNet18 Input Size	224 × 224 RGB
YOLO Input Size	640 × 640 RGB

■ Technology Mapping:

What?	Technology Used	Why?
Pitch Detection	YOLOv8 (ONNX)	Real-time single-stage object detection
Pitch Classification	ResNet18 (ONNX)	Transfer learning, 91.6% accuracy
Feature Extraction	OpenCV 4.12	Computer vision, efficient CPU processing
API Framework	FastAPI + Uvicorn	Async, auto-docs, Pydantic validation
Frontend	React 18 + Vite	Component-based, fast HMR, modern tooling
HTTP Client	Axios	Interceptors, promise-based, easy error handling
Database	MongoDB Atlas	Flexible schema, JSON-native, cloud-managed
Authentication	JWT + bcrypt	Stateless auth, secure password hashing
Payment	Razorpay	India-focused, easy integration, webhook support
AI Chatbot	Google Gemini	Free tier, strong cricket knowledge
Weather	OpenWeatherMap	Free tier, real-time data, lat/lon support
Deployment (BE)	Render / Railway	Free tier, auto-deploy from GitHub
Deployment (FE)	Vercel / Netlify	Free tier, Vite-optimized, CDN

■ OpenCV Features Summary:

Feature	Method	Output	Cricket Impact
Grass Coverage	HSV filtering (H: 25-85)	%, Level (High/Med/Low)	Grass = swing/seam
Crack Detection	Canny edges + contours	Count, Severity (H/M/L)	Cracks = spin turn
Moisture	HSV V-channel mean brightness	Score 0-100, Level	Wet = slow, Dry = spin
Color Profile	K-means clustering (k=3)	Type (Brown/Red/Green)	Prep type indicator
Texture	Laplacian variance	Roughness level	Rough = variable bounce
Brightness	Grayscale mean (0-255)	5 brightness levels	Hardness indicator

■■ Cricket Rule Engine Summary:

#	Condition	Action	Reason
R1	Grass > 60%	Bowling +/- 15%	Heavy grass aids swing
R2	Cracks = High	Spin +/- 20%	Deep cracks grip for spinners
R2b	Cracks = Medium	Spin +/- 12%	Moderate cracks help spin
R3	Very Dry + Grass < 20%	Seam +/- 15%	Dry pitch + seam movement
R4	Low cracks + Grass 20-50%	Batting +/- 10%	Balanced pitch
R5	Very Dry + High Cracks	Spin +/- 15%	Extreme turning conditions
R6	Wet/Damp + Grass > 50%	Bowling +/- 12%	Humid + grass = swing

■ **TIP: INTERVIEW STRATEGY:** Use the STAR method for behavioral questions (Situation, Task, Action, Result). For technical questions, always explain WHY you made choices, not just WHAT you built. Acknowledge limitations honestly — it shows maturity.

■■ **NOTE: CONFIDENCE BUILDER:** You built a full-stack AI application with 91.6% accuracy using YOLOv8, ResNet18, FastAPI, React, MongoDB, Razorpay, and Gemini AI. This demonstrates breadth and depth across the entire software development stack. Be proud of what you built!